MZ UCA Cloud Academy

AWS Solutions Architect Associate (SAA)

Learning Notes 📝

Identity & Access Management (IAM)

Building Secure Cloud Foundations

Prepared By

Zulu Mthulisi

Cloud Governance Architect

◈ INTRODUCTION

Identity and Access Management (IAM) is one of the most important services in AWS.

Before a user can launch an EC2 instance, create an S3 bucket, access a database, or modify cloud resources, AWS must first determine:

◆ Who is making the request?

◆ What permissions do they have?

◆ Should access be granted?

IAM provides the framework that answers these questions.

In modern cloud environments, security is no longer centered around physical infrastructure. Instead, security is centered around identity, permissions, and governance.

This is why IAM is often described as the foundation of AWS security.

◈ WHAT IS AWS IAM?

AWS Identity and Access Management (IAM) is a global AWS service that allows organizations to securely manage access to AWS resources.

IAM enables administrators to:

✔ Create users

✔ Create groups

✔ Create roles

✔ Assign permissions

✔ Enforce security controls

✔ Govern access across AWS environments

IAM helps ensure that only authorized individuals and services can access AWS resources.

◈ AUTHENTICATION VS AUTHORIZATION

A Solutions Architect must understand the difference between Authentication and Authorization.

Authentication

Authentication answers the question:

"Who are you?"

Examples:

◆ Username and Password

◆ MFA Verification

◆ Single Sign-On (SSO)

When authentication succeeds, AWS confirms the identity.

Authorization

Authorization answers the question:

"What are you allowed to do?"

Examples:

◆ Launch EC2 Instances

◆ Read S3 Objects

◆ Delete Resources

◆ Access Billing Information

Authorization occurs after authentication.

AWS first verifies identity and then evaluates permissions.

◈ IAM USERS

An IAM User represents an individual identity within AWS.

Examples include:

◆ Employees

◆ Administrators

◆ Developers

◆ Service Accounts

Each IAM User has unique credentials and permissions.

Characteristics of IAM Users

✔ Unique Identity

✔ Login Credentials

✔ Programmatic Access (Optional)

✔ Assigned Permissions

✔ Individual Accountability

Best Practices

✔ Create one user per person

✔ Enable MFA

✔ Use strong passwords

✔ Avoid sharing credentials

✔ Review permissions regularly

Real-World Example

A cloud team consists of:

◆ Grace

◆ Ndabezitha

◆ Vusi

◆ Boitumelo

Each team member should have their own IAM User account.

This ensures accountability and auditability.

◈ IAM GROUPS

IAM Groups simplify permission management.

A Group is a collection of IAM Users.

Instead of assigning permissions to individual users, permissions are assigned to the Group.

Users inherit permissions from the Group.

Example

Developers Group

├── User A

├── User B

├── User C

└── User D

When permissions are attached to the Developers Group, every member receives the same permissions automatically.

Benefits of Groups

✔ Easier Administration

✔ Consistent Permissions

✔ Reduced Errors

✔ Improved Governance

✔ Better Scalability

Common Enterprise Groups

◆ Developers

◆ Security Team

◆ Finance Team

◆ Administrators

◆ Operations Team

◈ IAM POLICIES

Policies are the mechanism AWS uses to define permissions.

Policies are written in JSON format and determine what actions are allowed or denied.

Policy Structure

Every policy contains:

Effect

Defines whether access is:

✔ Allow

OR

✖ Deny

Action

Specifies what can be performed.

Examples:

◆ ec2:StartInstances

◆ s3:GetObject

◆ iam:CreateUser

Resource

Defines which AWS resource can be accessed.

Examples:

◆ Specific S3 Bucket

◆ Specific EC2 Instance

◆ Entire AWS Account

Example Policy Logic

Allow:

s3:GetObject

Resource:

StudentNotesBucket

Result:

Users can view objects stored inside the bucket.

◈ POLICY EVALUATION

Whenever a request is made, AWS evaluates permissions.

Process:

1. User Authenticates

↓

1. AWS Evaluates Policies

↓

1. AWS Checks Deny Statements

↓

1. AWS Checks Allow Statements

↓

1. AWS Makes Final Decision

Critical Exam Rule

Explicit Deny Always Wins

If one policy allows access and another policy explicitly denies access, the request is denied.

Example

Policy A

Allow EC2 Access

Policy B

Deny EC2 Access

Result:

✖ Access Denied

AWS always prioritizes Explicit Deny.

◈ IAM ROLES

IAM Roles provide temporary permissions.

Unlike Users, Roles do not have permanent credentials.

Roles are assumed when access is required.

Why Roles Matter

Permanent credentials create security risks.

Roles eliminate many of these risks by providing temporary credentials.

Common Role Use Cases

EC2 → S3

An EC2 instance requires access to an S3 bucket.

Instead of storing access keys, an IAM Role is attached to the instance.

Lambda → DynamoDB

A Lambda function requires database access.

A Role grants temporary permissions.

Cross-Account Access

Organizations may need users from one AWS account to access another AWS account.

IAM Roles facilitate secure cross-account access.

Best Practice

Whenever possible:

Use Roles Instead of Access Keys

This is both an AWS recommendation and a common certification exam topic.

◈ USERS VS GROUPS VS ROLES

Component	Purpose
User	Represents an Identity
Group	Represents Multiple Users
Role	Provides Temporary Permissions

Quick Memory Trick

User = Person

Group = Team

Role = Temporary Job Assignment

◈ LEAST PRIVILEGE

Least Privilege is one of the most important security principles in cloud computing.

Definition

Provide users only the permissions required to perform their job.

No more.

No less.

Example

Developer Needs:

✔ Start EC2

✔ Stop EC2

✔ View Logs

Developer Does Not Need:

✖ Delete IAM Users

✖ Modify Billing

✖ Disable Security Controls

Benefits

✔ Reduced Security Risk

✔ Smaller Attack Surface

✔ Improved Compliance

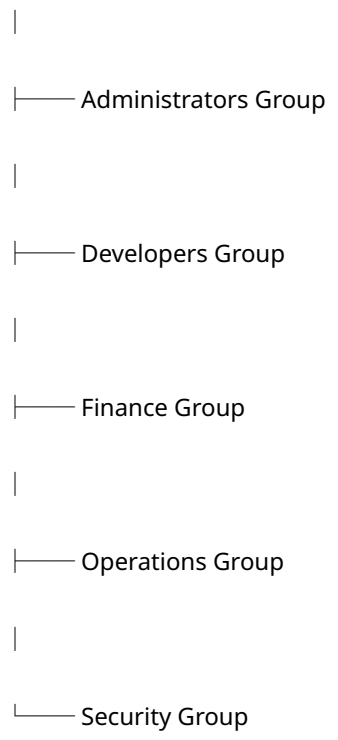
✔ Better Governance

✔ Stronger Security Posture

◈ ENTERPRISE IAM ARCHITECTURE

A mature AWS environment often follows this structure:

Root Account



Roles

◆ EC2 Role

◆ Lambda Role

◆ Backup Role

◆ Cross-Account Role

Benefits:

✔ Governance

✔ Scalability

✔ Separation of Duties

✔ Auditability

✔ Compliance

◈ IAM SECURITY BEST PRACTICES

AWS recommends the following controls:

Root Account

✔ Enable MFA

✔ Do Not Use Daily

✔ Secure Credentials

Users

✔ Strong Passwords

✔ MFA

✔ Least Privilege

✔ Credential Rotation

Permissions

✔ Use Groups

✔ Use Roles

✔ Regular Reviews

✔ Remove Unused Access

Monitoring

✔ CloudTrail

✔ Access Reviews

✔ Logging

✔ Security Audits

◈ AWS CERTIFICATION EXAM TIPS

Frequently Tested Concepts:

✔ IAM Users

✔ IAM Groups

✔ IAM Roles

✔ IAM Policies

✔ MFA

✔ Root Account Security

✔ Authentication

✔ Authorization

✔ Least Privilege

✔ Shared Responsibility Model

Remember

User = Identity

Group = Permission Management

Role = Temporary Access

Policy = Permission Document

Least Privilege = Security Principle

◈ THINK LIKE A CLOUD GOVERNANCE ARCHITECT

Before granting access, ask:

◆ Who needs access?

◆ Why do they need access?

◆ Which resources are required?

◆ How long is access needed?

◆ What is the minimum permission required?

◆ How will access be monitored?

◆ How will access be revoked?

These questions separate administrators from architects.

◈ FINAL TAKEAWAY

AWS IAM is not simply a security service.

It is the foundation of cloud governance.

Strong identity management enables:

✔ Security

✔ Compliance

✔ Accountability

✔ Operational Excellence

✔ Enterprise Scalability

The best cloud architects do not focus on granting access.

They focus on controlling access.

MZ UCA PRINCIPLE

"Anyone can create resources.

Architects design control.

Governance Architects design trust."

Zulu Mthulisi

Cloud Governance Architect

MZ UCA Cloud Academy